airlock.

Treasury controls for cross-chain payouts



Built on Portaldot V3

Makssay and Kuznets

Relay authority needs payout limits

A valid relay instruction can still
request an unwanted payout.

Who needs this

Teams holding treasury reserves on Portaldot
and accepting payout requests from other networks.

Airlock adds enforceable limits and an independent way to intervene.

Payout rules execute on Portaldot V3

01 Spending budget

The contract caps cumulative releases and preserves a reserve floor.

02 Quarantine

Larger requests wait. Funds remain reserved inside the treasury.

03 Guardian control

A separate guardian can pause releases or cancel queued requests.

Real requests and recorded results

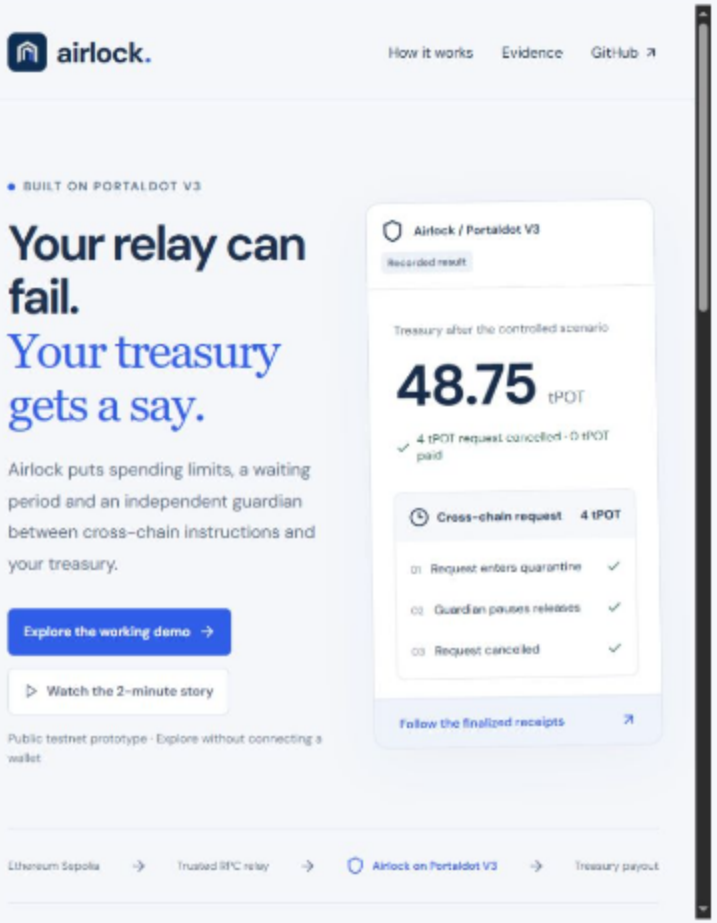
0.25 tPOT

paid in an earlier cross-testnet run

4 tPOT request

quarantined, paused and cancelled
0 tPOT paid during the controlled scenario

Recorded on Portaldot V3 testnet
Full transaction receipts in the repository



Portaldot holds the treasury and the rules

External request

A Sepolia outbox emits a payout instruction.
The trusted RPC relay checks the receipt
and waits for six canonical confirmations.

Portaldot execution

A Solidity contract runs through Revive.
The treasury holds native tPOT.
The guardian signs native Substrate calls.

Deployed on V3 testnet

Chain 420420777
0x3bbb62bf86d40f26a87e70fe56ee073e69198713

Next steps and continued ecosystem support

A reusable payout layer for Portaldot teams

Guardian alerts and a response demo inside quarantine
Additional public-testnet adversarial scenarios
Simpler setup and mainnet V3 compatibility checks

Seeking V3 technical feedback and pilot treasury teams

Public demo: airlock-v3-by-makssay-and-kuznets.vercel.app
Source: github.com/Makssay/AirLock
Video: youtu.be/LKbyCsM62rQ